



When Is a Scale Redundant? Data-Dependent Scale Redundancy and Zero-Training Pruning for Real-Time Object Detection

Yuanchao Yan^{a,**}, Xin Yan^a, Jianlong Liu^a

^aDepartment of Artificial Intelligence and Computational Science, Science and Technology College, Gannan Normal University, Ganzhou 341000, China

ABSTRACT

Real-time detectors run at full capacity on every image, yet where their computation is redundant is rarely quantified. We study YOLO26n on three datasets spanning opposite object-size regimes (VOC, COCO, VisDrone) and report two findings. First, scale redundancy is dataset-dependent: removing the small-object scale P3 costs 0.010/0.032/0.101 mAP on VOC/COCO/VisDrone, whereas the large-object scale P5 costs 0.417/0.176/ ≈ 0 ; per-scale gradient contributions track the small-object fraction, and P3 versus P5 importance reverses between 3.8% and 24.8% small objects (a three-point crossover near 12.6%, not a law). Second, FLOPs $\neq$ FPS: head-level cell skipping yields no GPU speedup, and the $\sim 95\%$ cell redundancy is only exploitable as a mask-level oracle, not as real skipping on GPU. Where a scale is truly redundant, dropping it is a zero-training optimization: on VisDrone, P5 removal cuts FLOPs/parameters by 10.4%/28.6% at identical mAP and reduces single-image latency by 6%. We also report negative results—early exit is infeasible and fine-tuning a dropped scale can hurt—as cautionary evidence for FLOPs-only efficiency claims.

© 2026 Elsevier Ltd. All rights reserved.

Keywords: real-time object detection, model efficiency, spatial token pruning, early exit, YOLO

1. Introduction

As LLMs that *overthink* spend tokens that do not improve the answer Wei et al. (2022); Feng et al. (2025), real-time detectors may *over-see*: they compute features the input does not need. Detectors are a clean testbed because their computation decomposes into explicit units at three granularities—grid cells (tokens), feature-pyramid scales, and model checkpoints.

Prior work leaves important gaps: token-pruning methods Rao et al. (2021); Liang et al. (2022); Bolya et al. (2023); Meng et al. (2021) report large FLOPs savings but target attention models and report FLOPs rather than latency; early-exit

detectors Teerapittayanon et al. (2016); Scholte et al. (2025); Menezes et al. (2025) attach confidence branches without identifying which features can be omitted; and efficiency studies are almost always single-dataset, leaving the dataset-dependence of redundancy unexamined.

This paper conducts a cross-distribution empirical study with YOLO26n Jocher et al. (2026) on VOC, COCO, and VisDrone, plus a cross-architecture check with YOLO11n, asking two questions. (RQ1) Is a detection scale redundant, and is the redundant scale stable across datasets? P3 is nearly redundant on VOC but critical on VisDrone, while P5 is completely redundant on VisDrone; gradient shares track the small-object fraction, and the P3/P5 ranking reverses between 3.8% and 24.8% small objects. (RQ2) Can redundancy become real speed? Cell-level redundancy is large (about 95% of grid cells) but, on GPU, only exploitable as a mask-level oracle—real block skipping neither speeds up inference nor preserves accuracy without re-training. Dropping a redundant scale is instead a zero-training, latency-reducing optimization compared against input-resolution scaling and a smaller model.

Contributions: (1) a cross-distribution per-scale contribution analysis showing the redundant scale is data-dependent and tracked by the small-object fraction, with a three-point trend

^{**}Corresponding author

e-mail: yanyuanchao@gnnustc.com (Yuanchao Yan),
yanxin@gnnustc.com (Xin Yan), liujianlong@gnnustc.com (Jianlong Liu)

(not a law) reversing the P3/P5 ranking around 12.6%; (2) an honest FLOPs-vs-FPS analysis showing cell-level redundancy is only a mask-level oracle on GPU, plus a zero-training scale-drop recipe compared against input-resolution scaling and a smaller model; (3) negative results (early exit infeasible, fine-tuning a dropped scale can hurt) as cautionary evidence.

2. Related Work

Dynamic token pruning and early exit. DynamicViT Rao et al. (2021), EViT Liang et al. (2022), ToMe Bolya et al. (2023), and AdaViT Meng et al. (2021) prune or merge tokens adaptively, reporting 30–40% FLOPs savings with negligible accuracy loss on attention models. Self-distillation of importance signals is central to TokenSkip Xia et al. (2025) and Self-Calibration Huang et al. (2025). BranchyNet Teerapittayanon et al. (2016), EOBranch Scholte et al. (2025), and AnytimeYOLO Menezes et al. (2025) attach early-exit branches to YOLO. These lines of work share our goal—spend computation only where it helps—but report FLOPs rather than latency, target attention models or exits, and do not identify which features a detector can actually omit; for a CNN detector we show the FLOPs claim does not survive head-level cell pruning.

Dynamic computation and pruning in detectors. Dynamic R-CNN Zhang et al. (2020) and YOLOv7 Wang et al. (2023) adapt training targets or label assignment during optimization, Sparse R-CNN Sun et al. (2021) and Cascade RPN Vu et al. (2019) reduce proposal-level computation, and channel-pruning recipes—Network Slimming Liu et al. (2017), AutoSlim Yu and Huang (2020), and popular YOLOv5 pruning tutorials Jocher et al. (2023)—remove channels with negligible accuracy loss, including MCCP Yan et al. (2023) for multi-collaboration channel pruning. Real-time detectors Redmon et al. (2016); Tian et al. (2019); Lin et al. (2017b); Jocher et al. (2026) rely on multi-scale pyramids Lin et al. (2017a), and cascade Cai and Vasconcelos (2018) or task-aligned Feng et al. (2021) designs assume all scales are needed. Two differences distinguish our work. Unlike channel-pruning methods such as Network Slimming Liu et al. (2017) or MCCP Yan et al. (2023), which find important channels by sparse training or search and then retrain, the proposed scale drop removes an entire feature-pyramid scale with zero training, because the redundant scale is identified from data statistics alone. Unlike dynamic-proposal detectors such as Sparse R-CNN Sun et al. (2021), which learn a small set of object proposals, we keep the dense grid and prune the pyramid’s native scales. Finally, we optimize latency (FPS) rather than FLOPs, showing where each type of saving actually transfers on GPU. On the deployment side, the cheapest generic speed-ups—reducing input resolution or removing the largest scale entirely—are common engineering practice; we treat them as first-order baselines in Sec. 4.5.

3. Method

3.1. Contribution analysis

For the FPN-style neck scales P_3, P_4, P_5 Lin et al. (2017a), the importance of scale i is measured by (i) gradient contribution $G_i = \|F_i \odot \partial L / \partial F_i\|_1$ (normalized), (ii) counterfactual loss

change $C_i = (L(Y^{-i}) - L(Y)) / L(Y)$, and (iii) detection-gain ablation $\Delta AP_i = AP(F) - AP(F_{-i})$ using a *fair* head-level mask: the pruned scale produces no detections (scores $\rightarrow -\infty$), avoiding false positives from naive zero-feeding.

3.2. Spatial token budget and self-distillation gate

Following adaptive token pruning Rao et al. (2021); Liang et al. (2022) and self-distillation of importance signals Xia et al. (2025); Huang et al. (2025), a gate g_ϕ is a per-scale 1×1 convolution producing cell scores; for budget b we keep the top- b fraction of cells per image and suppress the rest in the head. We compare an *oracle* (rank by the model’s own head scores), an *energy* gate (rank by $\|F_i\|_1$), and a *self-distilled* gate trained with the model frozen via importance-weighted BCE,

$$\begin{aligned} \mathcal{L}_{\text{gate}} &= \sum_i \text{BCE}(g_\phi(P_i), t_i) w_i, \\ t_i &= \max_c \sigma(s_{i,c}), \quad w_i = 1 + 30t_i. \end{aligned} \quad (1)$$

where $s_{i,c}$ denotes the classification score output by the teacher’s detect head (not the feature-map value) for cell i and class c , and t_i is its max-class confidence. The weight $w_i = 1 + 30t_i$ up-weights high-confidence cells (a cell with $t_i > 0.1$ receives at least $4\times$ the background weight); the gate is trained with Adam ($\text{lr} = 10^{-3}$, batch 64) for 5 epochs with the backbone, neck, and head frozen.

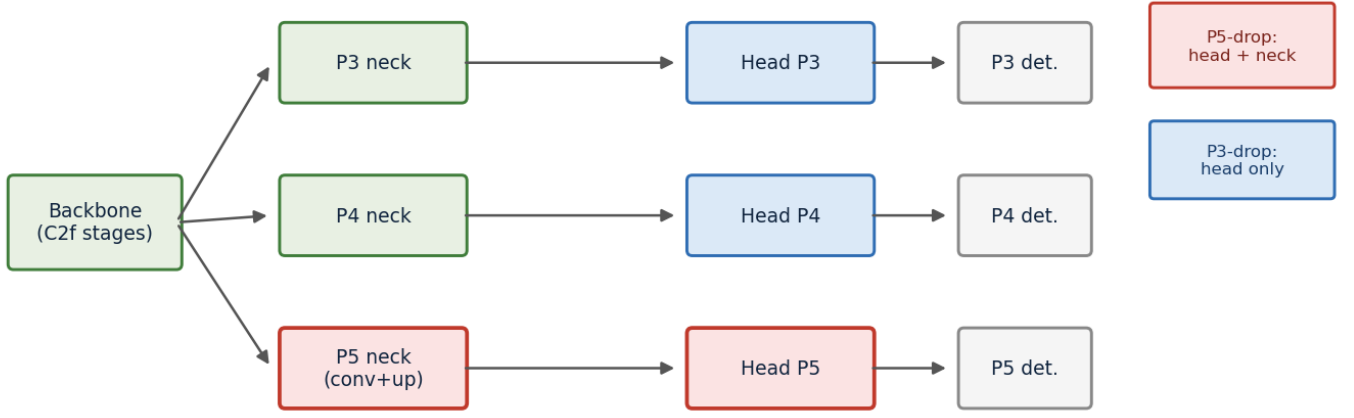
3.3. Zero-training scale drop

Unlike conventional channel pruning, which finds important channels through retraining or search Yan et al. (2023); Han et al. (2015), our scale drop needs no optimization: if a scale is redundant ($\Delta AP \approx 0$), it is removed and the remaining weights are transferred exactly, because removing the redundant tail branch does not change the input channels of the kept head scales. In practice we treat a scale as redundant when removal costs no more than ≈ 1 pp of mAP50-95: P5 on VisDrone costs ≈ 0 (unambiguous), while P3 on VOC costs 1.0 pp (0.010), a borderline case near the evaluation noise measured in Sec. 4.1. Scales costing more than 5 pp (P5 on VOC, 0.417; P3 on VisDrone, 0.101) are clearly necessary. The 1 pp threshold is a rule of thumb to be calibrated to the user’s accuracy budget, not a certified bound.

4. Experiments

4.1. Setup

We use YOLO26n Jocher et al. (2026), initialized from COCO-pretrained weights and trained with MuSGD (batch 32, image size 640, AMP) for 100 epochs on VOC Everingham et al. (2010) (16 551 train/4 952 test/2007) and VisDrone Zhu et al. (2022) (6 471 train/548 val); COCO Lin et al. (2014) uses the official pretrained weights on val2017. Full training details are in the supplementary material. Ablations use fair head-level masks with the standard validator; FLOPs are measured on fused models at 640. Latency is single-GPU FP32 on an NVIDIA GeForce RTX 5070 (CUDA 13.2, no TensorRT);



Zero-training: weights of kept branches transfer exactly; P5-drop also removes the P5 neck branch, so it saves far more parameters.

Fig. 1. What is dropped. P3-drop removes only the P3 branch of the detect head; P5-drop additionally removes the P5 neck branch (conv + upsampling), which is why it saves far more parameters and FLOPs.

we report batch-32 throughput (FPS) and batch-1 per-image latency (ms), since the two can rank optimizations differently. Evaluation is deterministic: three repeated VOC evaluations give identical mAP50-95 (0.5899), so the small per-scale differences we report (e.g., 0.010 on P3-VOC) are measurements rather than evaluation noise; training-seed variance is not separately quantified.

4.2. RQ1: scale importance is dataset-dependent

Table 1. Per-scale importance is monotonically tied to the small-object fraction s : gradient contribution G (cols. 3–4) and mAP cost ΔAP of removing a scale (cols. 5–6). The redundant scale flips between VOC (drop P3) and VisDrone (drop P5).

Dataset	small obj. (<32px)	G_{P3}	G_{P5}	ΔAP_{P3}	ΔAP_{P5}
VOC	3.8%	22%	39%	0.010	0.417
COCO	24.8%	28%	30%	0.032	0.176
VisDrone	85.9%	53%	7%	0.101	−0.0001

Fitting the three datasets (Fig. 2) gives $G_{P3}(s) = 0.494 + 0.094 \ln s$ and $G_{P5}(s) = 0.096 - 0.098 \ln s$ ($R^2 = 0.81/0.87$), crossing at $s^* \approx 12.6\%$; the mAP cost follows $\Delta AP_{P3} \approx 0.10 s^{0.73}$. With only three points these are trend descriptions, not laws: per-point residuals are 3.3/8.3/5.0 pp for G_{P3} and 2.6/6.7/4.1 pp for G_{P5} (VOC/COCO/VisDrone), and the crossover lies between the observed points at 3.8% and 24.8%, so 12.6% is an interpolation within our range, not an observed phenomenon. The ranking (P3 least, P5 most important) is also stable across model sizes and architectures on COCO (Table 2), so it is not a nano- or architecture-specific artifact.

The identity of the redundant scale is therefore transferable across model sizes on the same data: only the magnitude of the removal cost changes (e.g., ΔAP_{P5} grows from 0.176 on YOLO26n to 0.216–0.217 on YOLO26s/m), while the ranking and hence the drop decision are preserved.

Table 2. Per-scale importance is robust to model size and architecture on COCO val: the ranking $\Delta AP_{P3} < P4 < P5$ holds for YOLO26n/s/m and for YOLO11n, so the data-dependent scale-redundancy result is not specific to the nano model or to one architecture. G values are normalized gradient contributions; ΔAP uses the fair head-level mask.

Model	G_{P3}	G_{P4}	G_{P5}	ΔAP_{P3}	ΔAP_{P4}	ΔAP_{P5}
YOLO26n	28%	41%	30%	0.032	0.110	0.176
YOLO26s	33%	40%	27%	0.045	0.156	0.216
YOLO26m	32%	31%	38%	0.078	0.179	0.217
YOLO11n	35%	37%	28%	0.017	0.052	0.186

4.3. RQ2: cell-level redundancy is a mask-level oracle

Cell occupancy (cells containing object centers) is 0.05/0.19/0.74% (VOC) and 1.02/3.52/10.56% (VisDrone) for P3/P4/P5, yet an oracle that keeps the top-5% cells by head score retains essentially all mAP on both datasets (Table 3). This is a mask-level analysis: it suppresses scores without saving compute. Two observations explain why detection is this sparse. First, object centers cover a tiny fraction of the grid, and NMS only needs the few cells that vote for each object, so a score-based ranker needs almost no cells to reproduce the full output. Second, gate quality matters more than cell selection alone: a ground-truth gate that keeps only cells containing object centers reaches only 0.440/0.526 mAP at 10%/25% budget on VOC (supplementary material), well below the self-distilled gate (0.585/0.588), because detections require the score pattern around an object, not just its center cell.

4.4. RQ3: FLOPs $\neq$ FPS on GPU

The detect head is 17.3% of total FLOPs (0.90 of 5.21 G), and preprocessing+postprocessing account for 41% of end-to-end latency at batch 32 (batch-1 model latency 9.4 ms). We implemented real block-skipping of pruned cells (halo-padded gather $\rightarrow$ head convs $\rightarrow$ scatter); batch-32 throughput is flat or lower at every budget, so the FLOPs saved never materialize as speed (Table 4):

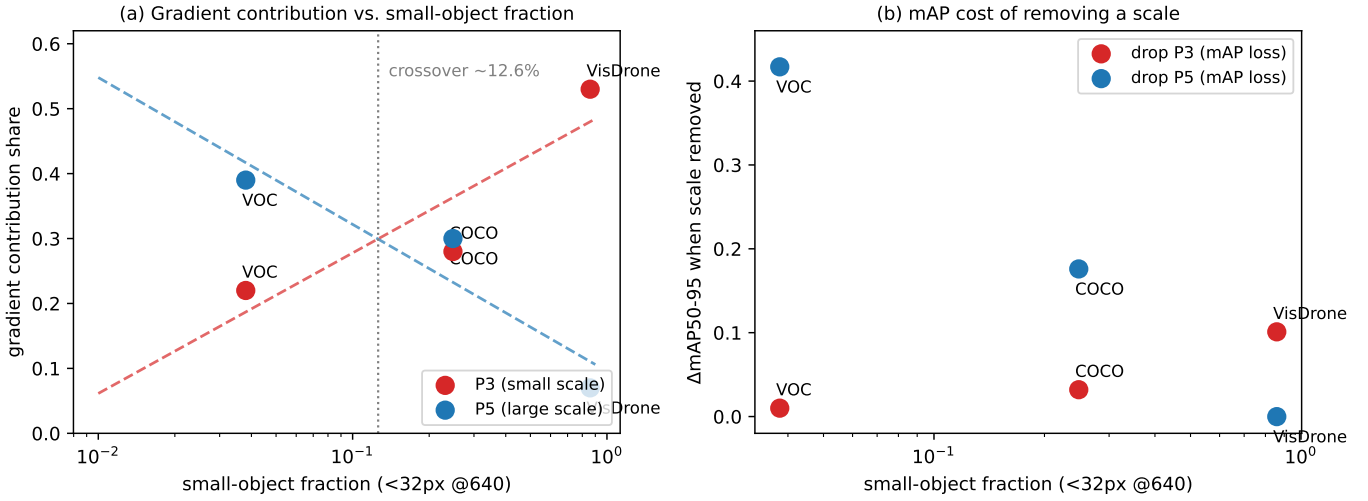


Fig. 2. Gradient contribution (a) and mAP cost of scale removal (b) versus small-object fraction; the P3/P5 ranking reverses in our three-point trend.

Table 3. Budget curves (mAP50-95, mask-level): an oracle ranking cells by the model’s own head scores retains essentially all mAP at 5% of cells; the self-distilled gate is within 0.5 pp of full accuracy at 10% budget (VOC: 0.5852 vs. 0.5899; VisDrone: 0.1719 vs. 0.1725) and 1.1 pp at 5%; the energy gate degrades much more on dense VisDrone. Full 5–75% curves and YOLO26s/26m ablations are in the supplementary material; these numbers assume full features (real skipping: Table 4).

Budget (% cells)	VOC			VisDrone		
	Oracle	Energy	Distilled	Oracle	Energy	Distilled
5%	0.5895	0.171	0.5789	0.1725	0.075	0.1675
10%	0.5896	0.298	0.5852	0.1725	0.109	0.1719
25%	0.5896	0.485	0.5882	0.1725	0.148	0.1724

Table 4. Head-level block skipping neither accelerates inference nor preserves accuracy: halo padding inflates compute and gather/scatter overhead cancels the FLOPs savings (throughput is flat or lower at every budget, 995→991/986/962/911 FPS), and even the halo-only 100%-budget variant is 19% slower (805 FPS). The accuracy gap to the mask-level oracle (Table 3) reflects the head seeing pruned features that differ from the full features used to train the gate.

Config	mAP50-95	GFLOPs	b32 FPS
Full	0.5899	5.21	995
Block-skip 5%	0.381	4.79	991
Block-skip 10%	0.421	4.83	986
Block-skip 25%	0.478	4.94	962
Block-skip 50%	0.513	5.12	911
Block-skip 100% (halo only)	0.520	5.50	805

4.5. Zero-training scale drop

We do not pursue difficulty routing or early exit as speed-ups (Sec. 5).

5. Negative results and lessons

Backbone early exit is infeasible on VOC: detection from P3-only or P3+P4 features yields 0.022/0.159 mAP, because deep features are the detection representation itself. Head-level token skipping does not accelerate GPUs (Table 4), a cautionary result

Table 5. Zero-training scale drop keeps mAP identical where the scale is redundant while cutting FLOPs/params and single-image latency; 30-epoch fine-tuning did not improve over transfer-only (0.169 vs. 0.173). P3-drop removes only the P3 head, whereas P5-drop also removes the P5 neck branch (Fig. 1), which is why the savings differ.

	mAP50-95	GFLOPs	Params(M)	b1 latency (ms)	b1 FPS
<i>VOC: drop P3 (head-only, zero training)</i>					
Full	0.5899	5.21	2.38	9.42	106
P3-drop	0.5806	4.93	2.36	8.43	119
<i>VisDrone: drop P5 (zero training)</i>					
Full	0.1725	5.20	2.38	10.02	100
P5-drop	0.1725	4.66	1.70	9.40	106

Table 6. Scale drop vs. input-resolution scaling. On VisDrone (85.9% objects <32 px), resolution scaling loses 2.8–3.5 pp because it destroys small-object detail, while P5-drop keeps mAP identical and cuts latency by 6%; on VOC, where small objects are rare, resolution scaling is cheaper.

	mAP50-95	GFLOPs	b32 FPS	b1 latency (ms)
<i>VOC (small-object fraction 3.8%)</i>				
Full @640	0.5899	5.21	603	9.42
Full @512	0.5759	—	1033	—
Full @480	0.5710	—	1114	—
P3-drop @640	0.5806	4.93	769	8.43
<i>VisDrone (small-object fraction 85.9%)</i>				
Full @640	0.1725	5.20	634	10.02
Full @512	0.1450	—	910	—
Full @480	0.1378	—	1009	—
P5-drop @640	0.1725	4.66	670	9.40

for FLOPs-only claims Rao et al. (2021); Liang et al. (2022); even the halo-only variant that skips no cells is 19% slower than the baseline, so the gather/scatter and halo-padding overhead alone exceeds any possible saving in this head. Fine-tuning a redundant-scale drop can also hurt (0.169 vs. 0.173).

6. Discussion

Why is scale redundancy data-dependent? In a feature pyramid, the small-object scale P3 operates at the highest resolution and carries the fine detail that small objects need, while

the deep scale P5 carries the most semantic, lowest-resolution features. When a dataset is dominated by large objects (VOC, 3.8% small), the high-resolution P3 features are barely used by the head and can be dropped with almost no loss; when it is dominated by small objects (VisDrone, 85.9% small), P3 is the critical scale and the redundant one is instead P5. The gradient contributions in Table 1 reflect exactly this allocation of the loss to scales, which is why the small-object fraction s is a usable (if imperfect) predictor of the redundant scale. The three-point fit should therefore be read as a diagnostic trend for a given architecture, not as a universal law: other architectures that allocate gradients differently (e.g., transformer-based necks) may shift the crossover.

The negative results also carry a general lesson for efficiency research on CNN detectors. Head-level token pruning reports FLOPs savings that do not transfer to GPU throughput, because dense convolutions cannot skip arbitrary cells without gather/scatter overhead and halo padding; attention models, by contrast, prune tokens natively. Reporting latency (batch-1 and batch-32) alongside FLOPs, as we do, is therefore essential, and we hope the community adopts latency reporting as a default in detector efficiency papers.

Finally, the zero-training recipe is deliberately conservative: it only removes a scale when the data already shows it is redundant, so no accuracy is traded for speed. This distinguishes it from retraining-based pruning Yan et al. (2023); Han et al. (2015), which can remove more compute at the cost of an optimization loop; the two are complementary, and the scale drop can be composed with channel pruning as a first, training-free step.

7. Conclusion

Detectors do over-see, but where the redundancy lives is dictated by the data. Across VOC, COCO, and VisDrone, the redundant scale is dataset-dependent—P3 is nearly redundant on VOC and critical on VisDrone, while P5 is completely redundant on VisDrone—and gradient contributions track the small-object fraction, with the P3/P5 ranking reversing between 3.8% and 24.8% small objects (a three-point trend, not a law). Cell-level redundancy is large but, in our measurements, only a mask-level oracle: real block skipping neither speeds up GPU inference nor preserves accuracy without re-training. Where a scale is truly redundant, dropping it is zero-training and cuts single-image latency by 6–12% at identical mAP—the right choice where the cheap generic alternative fails: on dense small-object data (VisDrone), resolution scaling loses 2.8–3.5 pp while P5-drop loses nothing.

Equally important is what does not work: backbone early exit is infeasible, and fine-tuning a redundant-scale drop can hurt. These negative results argue for dataset-aware efficiency and for reporting latency, not just FLOPs. Limitations include three datasets and models up to YOLO26m (plus a YOLO11n architecture check); the $\ln s$ trend and the 12.6% crossover are empirical fits that must be re-validated on remote sensing, medical, or industrial data. Edge-deployment latency (Jetson/TensorRT or mobile SoCs) is explicitly out of scope: all latency numbers

are desktop-GPU (RTX 5070), and measuring the zero-training recipe on real edge hardware is left to future work.

From a practitioner’s perspective the recipe is simple: estimate the small-object fraction of the target data, use the fitted trend (or a single scale-removal probe) to identify the least important scale, remove it, and reuse the remaining weights without training. Because the ranking is stable across model sizes and architectures (Table 2), the decision transfers between YOLO variants on the same data. We leave three extensions to future work: (i) validating the $\ln s$ trend on more diverse distributions, such as remote sensing, medical imaging, and industrial inspection, where the small-object fraction can exceed 95% or drop below 1%; (ii) measuring the zero-training recipe on edge hardware (Jetson/TensorRT), where batch-1 latency and power matter more than desktop throughput; and (iii) extending the per-scale analysis to transformer-based detectors such as RT-DETR, whose token-based computation may make the mask-level oracle exploitable in practice.

CRedit authorship contribution statement

Yuanchao Yan: Conceptualization, Methodology, Software, Validation, Writing – original draft. **Xin Yan:** Investigation, Data curation, Validation. **Jianlong Liu:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All datasets used in this study (PASCAL VOC, MS COCO, and VisDrone) are publicly available from their official sources. Code and models are available at <https://github.com/Noisercontrollers/yolo-scale-drop>.

References

- Bolya, D., Fu, C.Y., Dai, X., Zhang, P., Feichtenhofer, C., Hoffman, J., 2023. Token merging: Your ViT but faster, in: International Conference on Learning Representations (ICLR).
- Cai, Z., Vasconcelos, N., 2018. Cascade R-CNN: Delving into high quality object detection, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
- Everingham, M., Van Gool, L., Williams, C.K.I., Winn, J., Zisserman, A., 2010. The Pascal Visual Object Classes (VOC) challenge. International Journal of Computer Vision 88 (2), 303–338.
- Feng, C., Zhong, Y., Gao, Y., Scott, M.R., Huang, W., 2021. Toood: Task-aligned one-stage object detection, in: IEEE/CVF International Conference on Computer Vision (ICCV).
- Feng, S., Fang, G., Ma, X., Wang, X., 2025. Efficient reasoning models: A survey. Transactions on Machine Learning Research .
- Han, S., Pool, J., Tran, J., Dally, W., 2015. Learning both weights and connections for efficient neural networks, in: Advances in Neural Information Processing Systems (NeurIPS).
- Huang, Y., et al., 2025. Self-calibration for efficient reasoning. arXiv preprint arXiv:2503.00210 .

- Jocher, G., Qiu, J., Liu, M., Lyu, S., Akyon, F.C., Kalfaoglu, M.E., 2026. Ultralytics yolo26: Unified real-time end-to-end vision models. doi:10.48550/arXiv.2606.03748, arXiv:2606.03748.
- Jocher, G., et al., 2023. Ultralytics YOLOv5. <https://github.com/ultralytics/yolov5>.
- Liang, Y., Ge, C., Tong, Z., Song, Y., Wang, J., Xie, P., 2022. Evit: Expediting vision transformers via token reuse, in: International Conference on Learning Representations (ICLR).
- Lin, T.Y., Dollár, P., Girshick, R., He, K., Hariharan, B., Belongie, S., 2017a. Feature pyramid networks for object detection, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
- Lin, T.Y., Goyal, P., Girshick, R., He, K., Dollár, P., 2017b. Focal loss for dense object detection, in: IEEE/CVF International Conference on Computer Vision (ICCV).
- Lin, T.Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., Zitnick, C.L., 2014. Microsoft COCO: Common objects in context, in: European Conference on Computer Vision (ECCV), Springer. pp. 740--755.
- Liu, Z., Li, J., Shen, Z., Huang, G., Yan, S., Zhang, C., 2017. Learning efficient convolutional networks through network slimming, in: IEEE International Conference on Computer Vision (ICCV), pp. 2736--2744.
- Menezes, R., Filgueiras, E., Silveira, J., et al., 2025. AnytimeYOLO: Early exits for interruptible object detection. arXiv preprint arXiv:2503.17497.
- Meng, L., Li, H., Chen, B.C., Lan, S., Wu, Z., Jiang, Y.G., Lim, S.N., 2021. AdaViT: Adaptive vision transformers for efficient image recognition, in: IEEE/CVF International Conference on Computer Vision (ICCV).
- Rao, Y., Zhao, W., Liu, B., Lu, J., Zhou, J., Hsieh, C.J., 2021. DynamicViT: Efficient vision transformers with dynamic token sparsification, in: Advances in Neural Information Processing Systems (NeurIPS).
- Redmon, J., Divvala, S., Girshick, R., Farhadi, A., 2016. You only look once: Unified, real-time object detection, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
- Schulte, D., Zwemer, M.H., Bondarev, E., 2025. Accelerating YOLO with EOBranch: An early exit approach for adaptive object detection, in: International Conference on Image Analysis and Processing (ICIAP).
- Sun, P., Zhang, R., Jiang, Y., Kong, T., Xu, C., Zhan, W., Tomizuka, M., Li, L., Yuan, Z., Wang, C., et al., 2021. Sparse R-CNN: End-to-end object detection with learnable proposals, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 14454--14463.
- Teerapittayanon, S., McDanel, B., Kung, H.T., 2016. BranchyNet: Fast inference via early exiting from deep neural networks, in: International Conference on Pattern Recognition (ICPR).
- Tian, Z., Shen, C., Chen, H., He, T., 2019. FCOS: Fully convolutional one-stage object detection, in: IEEE/CVF International Conference on Computer Vision (ICCV).
- Vu, T., Jang, H., Pham, T.X., Yoo, C.D., 2019. Cascade RPN: Delving into high-quality region proposal networks with adaptive convolution, in: Advances in Neural Information Processing Systems (NeurIPS).
- Wang, C.Y., Bochkovskiy, A., Liao, H.Y.M., 2023. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors, in: IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 7464--7475.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q.V., Zhou, D., et al., 2022. Chain-of-thought prompting elicits reasoning in large language models, in: Advances in Neural Information Processing Systems (NeurIPS).
- Xia, H., Li, Y., Leong, C.T., Wang, W., Li, W., 2025. TokenSkip: Controllable chain-of-thought compression in llms. arXiv preprint arXiv:2502.12067.
- Yan, Y., Liu, B., Lin, W., Chen, Y., Li, K., Ou, J., Fan, C., 2023. MCCP: Multi-collaboration channel pruning for model compression. Neural Processing Letters 55 (3), 2777--2797.
- Yu, J., Huang, T., 2020. AutoSlim: Towards one-shot architecture search for channel numbers, in: International Conference on Machine Learning (ICML), pp. 10862--10872.
- Zhang, H., Chang, H., Ma, B., Wang, N., Chen, X., 2020. Dynamic R-CNN: Towards high quality object detection via dynamic training, in: European Conference on Computer Vision (ECCV), Springer. pp. 260--275.
- Zhu, P., Wen, L., Du, D., Bian, X., Ling, H., Hu, Q., Nie, J., Cheng, H., Liu, C., Ma, X., et al., 2022. Detection and tracking meet drones challenge. IEEE Transactions on Pattern Analysis and Machine Intelligence 44 (11), 7380--7399.